Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (2)

Roll No : 496

Name : Dhruv Limbasiya

Enrollment : 24031701027

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

In [1]:
```
class Rectangle:
    def __init__(self , l , b):
        self.Length = l
        self.Breadth = b

    def Area(self):
```

```
        return self.Length * self.Breadth
r1 = Rectangle(10 , 5)
print(r1.Area())
```

```
50
```

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

In [25]:
```
class Square:
    def __init__(self , l ):
        self.Length = l

    def Area(self):
        return self.output(self.Length**2)

    def output(self ,value):
        print(value)

length = int(input("Enter Length of Square:"))
r1 = Square(length)
r1.Area()
```

```
Enter Length of Square: 5
25
```

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

In [18]:
```
class Rectangle:
    def __init__(self , l , b):
        self.Length = l
        self.Breath = b

    def Area(self):
        return self.output(self.Length * self.Breath)

    def output(self ,value):
        if self.Length == self.Breath:
            print("This is Square")
        print(value)

length = int(input("Enter Length of Reactangle:"))
breath = int(input("Enter Breath of Reactangle:"))
r1 = Rectangle(length , breath)
r1.Area()
length = int(input("Enter Length of Reactangle:"))
breath = int(input("Enter Breath of Reactangle:"))
r2 = Rectangle(length ,breath)
r2.Area()
```

```
Enter Length of Reactangle: 10
Enter Breath of Reactangle: 5
50
Enter Length of Reactangle: 5
Enter Breath of Reactangle: 5
This is Square
25
```

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to accees the private attribute from outside of the class.

In [27]:
```
class Square:
    def __init__(self , l ):
        self.__size = l
```

```
    def get_side(self):
        print(self.__size)

    def set_side(self , l):
        self.__size = l

    def Area(self):
        return self.output(self.__size**2)

    def output(self ,value):
        print(value)

length = int(input("Enter Length of Square:"))
r1 = Square(length)
r1.Area()
r1.get_side()
r1.set_side(6)
r1.get_side()
```

```
Enter Length of Square: 5
25
5
6
```

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balanace. It has two methods getBalance() and printBalance().

In [28]:
```
class Profit:
    def getProfit(self):
        self.profit = int(input("Enter profit amount: "))
        return self.profit


class Loss:
    def getLoss(self):
```

```
        self.loss = int(input("Enter loss amount: "))
        return self.loss


class BalanceSheet(Profit, Loss):
    def getBalance(self):
        p = self.getProfit()
        l = self.getLoss()
        self.balance = p - l
        return self.balance

    def printBalance(self):
        print("Final Balance:", self.balance)


obj = BalanceSheet()
obj.getBalance()
obj.printBalance()
```

```
Enter profit amount:  1000
Enter loss amount:  400
Final Balance: 600
```

15) WAP to demonstrate all types of inheritance.

In [ ]:

```
print("Single Inheritance")
class A:
    def show_a(self):
        print("Class A")

class B(A):
    def show_b(self):
        print("Class B")

obj1 = B()
obj1.show_a()
obj1.show_b()

print("\nMultilevel Inheritance")
class C(B):
```

```
    def show_c(self):
        print("Class C")

obj2 = C()
obj2.show_a()
obj2.show_b()
obj2.show_c()

print("\nHierarchical Inheritance")
class D(A):
    def show_d(self):
        print("Class D")

class E(A):
    def show_e(self):
        print("Class E")

obj3 = D()
obj4 = E()
obj3.show_a()
obj3.show_d()
obj4.show_a()
obj4.show_e()

print("\nMultiple Inheritance")
class F:
    def show_f(self):
        print("Class F")

class G:
    def show_g(self):
        print("Class G")

class H(F, G):
    def show_h(self):
        print("Class H")

obj5 = H()
obj5.show_f()
obj5.show_g()
obj5.show_h()
```

```
print("\nHybrid Inheritance")
class I(A):
    def show_i(self):
        print("Class I")

class J(A):
    def show_j(self):
        print("Class J")

class K(I, J):
    def show_k(self):
        print("Class K")

obj6 = K()
obj6.show_a()
obj6.show_i()
obj6.show_j()
obj6.show_k()
```

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the super() and then initialize the salary attribute.

In [6]:
```
class Person :
    def __init__(self , name , age):
        self.name = name
        self.age = age

    def display(self):
        return f"Name: {self.name}, Age: {self.age}"

class Employee(Person):
    def __init__(self , name , age , salary):
        super().__init__(name , age)
```

```
        self.salary = salary

    def display(self):
        return f"Name: {self.name}, Age: {self.age}, Salary: {self.salary}"

p1 = Person("dhruv", 30)
print(p1.display())
e1 = Employee("rajarshi", 40, 50000)
print(e1.display())
```

```
Name: dhruv, Age: 30
Name: rajarshi, Age: 40, Salary: 50000
```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [5]:
```
class Shape:
    def draw(self):
        raise NotImplementedError("Subclass must implement draw()")

class Rectangle(Shape):
    def draw(self):
        print("Drawing a Rectangle")

class Circle(Shape):
    def draw(self):
        print("Drawing a Circle")

class Triangle(Shape):
    def draw(self):
        print("Drawing a Triangle")

shapes = [Rectangle(), Circle(), Triangle()]
```

```
for shape in shapes:
    shape.draw()
```

```
Drawing a Rectangle
Drawing a Circle
Drawing a Triangle
```

In [ ]: